

What other aspects of software design is essential for an engineer to consider?

⇒ Good organization of software

⇒ Object-oriented programming; all about ensuring that the real world is clearly modeled.

⇒ Performance ⇒

- (1) Time
- (2) Memory
- (3) Energy

⇒ Security ✓

How can a program optimize memory?

⇒ Dynamic memory allocation vs static

⇒ Use data types that consume less memory.

char ⇒ 1 byte

Sort an array of numbers, where the numbers are between 0 and 255.

```

void sort () {
    int *array = new int[arraylen];
    cin >> arraylen;
    for (int i=0; i<arraylen; i++) {
        cin >> array[i];
    }
    for (int i=0; i<arraylen; i++) {
        for (int j=1; j<arraylen-i-1; j++) {
            if (array[i] > array[j]) {
                int temp = array[j];
                array[j] = array[i];
                array[i] = temp;
            }
        }
    }
}

```

8 bytes = 64 bits
 $- 2^{63}$ to $2^{63} - 1$

char *array = new int[arraylen];

1 byte = 8 bits

All chars store values from 0 to $2^8 - 1$
 $= 255$

Optimization of execution Time

1) Sorting algorithm $\Rightarrow$ make it more efficient;
i.e. an algorithm with lower time complexity.

But first you need to identify which part
of the code consumes more time.

Identifying is called profiling of code.

$\sim \> gcc \quad abc.c \quad -g$

$\sim \> gprof \quad a.out$

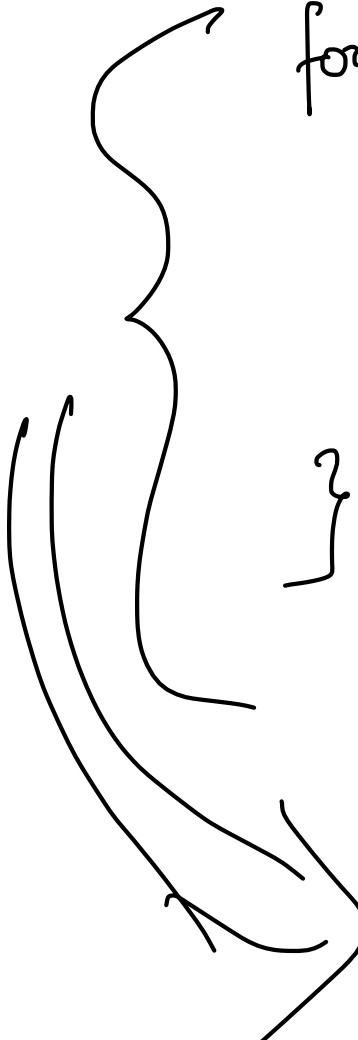
Gives you the time spent by the program in
each line of code.

DO not optimize prematurely; first identify
the portions that are slowing down the program.

② Use compiler optimizations

$\left. \begin{array}{l} \text{gcc -O1} \\ \text{gcc -O2} \end{array} \right\}$ makes code faster

```
int x[100];  
for (int i = 0; i < 25; i++) {  
    int j = i * 4;  
    x[j] = 5;  
}
```

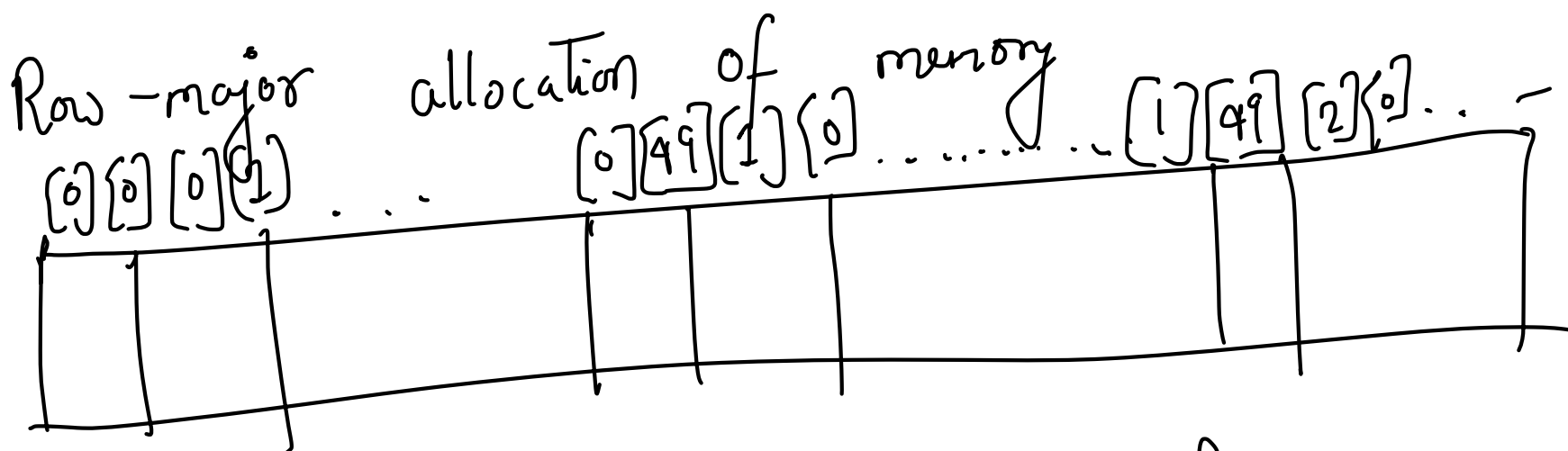


```
for (int j = 0; j < 100; j += 4) {  
    x[j] = 5;  
}
```

~~0~~
 $x[:, :] = 0;$

```
int x[100][50];
```

```
for (int j=0; j<50; j++) {  
    for (int i=0; i<100; i++) {  
        x[i][j] = 0;  
    }  
}
```



Memory hierarchy says that the type of memory that is physically closer to the processor is both fastest and the smallest in size.

Principle of locality $\Rightarrow$ Most of the content in memory ~~are~~ is close to each other.

A compiler should be able to optimize this code.

② Use compiler optimizations whenever execution time is

important. Try to follow principle of locality.

③ Try to use integer operations as much as possible instead of floating point operations,

int a, b; // change this to "float a, b;"

cin >> a >> b;

if (a > b) {
:
}

Float operations are much slower,

What is the cost of using optimization flags??

⇒ Higher compilation time

④ Utilize parallelism

⇒ Create threads (MIMD)

⇒ Vector operations (SIMD)

⇒ On modern CPUs, there are vector instructions

⇒ CPUs can do a single operation on large data in a single step.